Encoder-based LLMs: Building QA systems and Comparative Analysis

Saket Chaudhari[1] and Shalini Dangi[2]

Department of Computer Science and Engineering,
Indian Institute of Information Technology Surat,
Kholvad Campus. Kamrej Surat, Gujarat, 394190, India.

Abstract

Introduction of deep learning Large Language Models(LLMs) have caused significant development in fields of Natural Language Processing(NLP) and Computer (Vision)CV. This paper focuses on using the Encoder-based LLMs in building of Question-Answering (QA) systems and perform a comparative analysis of the evaluation results. In this paper we have studied about the existing solutions of the problem statement and their limitations. The paper uses BERT as the foundation model and other variants of it which includes RoBERTa, DistilBERT, ALBERT and XLM-RoBERTa. The paper provides comprehensive details about pretraining of all the models. Further paper shows finetuning of these pretrained model on a large reading comprehension dataset to build a QA system and evaluating it performance. The implementation of finetuning is done on cloud-based NVIDIA GPU Tesla T4 provided by Google Colab.The evaluated performances are then compared and inference is conducted. Inference with the help of chart shows the unique advantages and developments of each model along with certain limitations of the model. We also have worked towards optimizing the finetuning process in order to get best results. We achieve F1-score of models as follows: BERT(76.79), RoBERT(83.05), DistilBERT(68.98), ALBERT(81.55) and XLM-RoBERTa(78.73).

1. Introduction

Large Language Model(LLM) is a foundational deep learning model which can be used for various Natural Language Processing(NLP) and Computer Vision(CV) tasks. LLMs are based on the concept of Transfer Learning. They are first pretrained on massive amounts of unlabelled data and then can be finetuned easily with a labelled dataset on any downstream task. Encoder refers to the encoder part of the transformer architecture used by the model. In this paper we use the Bidirectional Encoder Representation of Transformer(BERT) encoder model as a foundation model and its variants which was launched by Google AI to build a Question-Answering(QA) system. The variants to be used include RoBERTa(Robustly optimized BERT approach), DistilBERT(Distilled version of BERT), ALBERT(A Lite BERT) and XLM-RoBERTa(Cross-lingual Language Model RoBERTa).

In order to perform a complex NLP task having sequential data, Recurrent Neural Networks(RNN) and Long-Short Term Memory Networks(LSTM) were used as state-of-art approaches [3]. They struggle with long-term dependencies and vanishing

gradient problems [2]. Further they do not use transfer learning and hence need to be trained from scratch which costs in terms of time and computational efficiency. These models along with transfer learning based models like ELMo, ULMFiT are unidirectional in nature and have task-specific architecture. These limitations cause poor performances on various NLP tasks.

BERT provides a solution to these limitations by introducing a bidirectional training approach [1] which reads the context in both directions and hence helps in deeper understanding of the language context. BERT is based on attention mechanism and hence supports parallel computation. And since it uses transfer learning it does not need to be trained from scratch, it can be easily fine-tuned on any task provided with a limited labelled dataset. Thus BERT overcomes the limitations caused by the previous models and gives state-of-art results on complex NLP tasks.

2. Related Work

There exist solutions for building complex QA systems and NLP systems in general. With the introduction of RNN it has become easier to handle sequential data and time-series data. It has an internal memory which helps it to memorize previous inputs and makes it suitable for NLP applications [2]. It has its limitations, as a solution LSTMs are introduced. LSTM is a type of RNN architecture designed to address the limitations of traditional RNNs in capturing long-term dependencies in sequential data. The key innovation in LSTM is the introduction of special memory cells and gating mechanisms that enable it to store and access information over long periods of time.
Attention mechanism is introduced to solve the limitations of RNNs which include lack of parallelism and long range dependencies. The attention mechanism is a technique used in deep learning models, particularly in NLP and CV tasks, to enable the model to focus on specific parts of the input data that are important to the current prediction or decision.

The transfer learning mechanism introduced helped in reducing the computational cost of training the model from scratch and helped to transfer features from one task to another. The transformer architecture [3] introduced in 2017 includes self-attention mechanisms, multi headed attention, positional encoding and multiple transformer layers. The precursor models used before was BiDAF model and ELMo model which attained a F1-score of 62.09 and 66.251 respectively on SQuAD 2 dataset [5].We will be using the BERT model and its variants which uses the encoder part of transformer architecture for pretraining and later fine-tuning model on task-specific dataset. The F1-score of BERT on SQuAD 2 dataset is 75.43 which makes it the most appropriate model for use in the project.

3. Task and Dataset

3.1 Task

The task implemented by the BERT models is **Question-Answering(QA)**. QA is an important NLP task in which the user asks a question in natural language to the model

as input and the model provides the answer in natural language as output. In order t0 produce optimal results the QA system uses deep learning and machine learning techniques in addition to NLP. The system when provided with a context paragraph and a question related to it, would answer correctly by extracting the answer from the given context.

QA systems have the following applications: Chatbot, Virtual assistant, Education tutor and Customer support. We will be building an Answer Retrieval QA system in this project.

3.2 Dataset

SQuAD 2 stands for Stanford Question Answering Dataset [5]. It is an update to the original SQuAD dataset, which was published in 2016. It is a dataset of reading comprehension questions and answer pairs derived from Wikipedia articles. The answers to all of the questions are provided in the form of a text span. The Question Answering model is trained on the train dataset and evaluated on the validation dataset.

Each examples includes of the following entries:
['id', 'title', 'context', 'question', 'answers']

SQuAD dataset consists of 2 parts:
- Train dataset - 130319 examples
- Validation dataset - 11873 examples

4. Pretraining of Models

Pretraining is a part of transfer learning, in which the model is trained on a large amount of unlabelled data in an unsupervised fashion. By following the training procedure model learns deeper language context by analysis of the data, finding features, semantic relationships and hidden patterns in the data. The main goal of pretraining is to allow the model to easily finetune on any NLP task with just a limited amount of labelled dataset hence, reducing the time and computational cost. Pretraining further reduces the overall cost in terms of computation infrastructure used and reduces the carbon footprint caused by the model.

The training procedure requires use of multiple hardware accelerators like Graphics Processing Units(GPUs) and Tensor Processing Units(TPUs). The most used hardware accelerators include NVIDIA 32GB V100 GPUs and 16GB V100 GPUs along with the Cloud TPUs provided by Google. The training time requires around a few days to weeks for completion depending on the hardware used for training. For instance BERT-base model is trained on 8 NVIDIA V100 GPUs for 12 days (8 * V100 * 12).

Model	BERT	ROBERTa	DistilBERT	ALBERT	XLM RoBERTa
Parameters	110M	125M	66M	11M	125M
L/H/A	12/768/12	12/768/12	6/768/12	Varies	12/768/12
Training Data	BooksCorpus + EnglishWikipedia =16GB	BERT + Ccnews+ Openwebtext+ stories=160 GB	BooksCorpus+ EnglishWikipedia =16GB	BooksCorpus+ EnglishWikipedia =16GB	Filtered Common Crawl data = 2.5TB

Table1: Pretraining information about BERT model variants

BERT - Bidirectional Encoder Representations from Transformers [1]
Pretraining Approach: Masked Language Modelling(MLM), Next Sentence Prediction(NSP)
MLM is also referred to as cloze task, in MLM 15% of the word sequence that is given to BERT as input are randomly masked with a [MASK] token. The model has to predict the masked words based on the other unmasked words in the context. In the NSP training process, the model receives pairs of sentences from a large monolingual corpus as input and learns to predict if the second sentence in the pair follows the first sentence.

RoBERTa - Robustly optimized BERT approach [6]
Pretraining Approach: Dynamic Masking(DM), No NSP
RoBERTa is a replication study of BERT which uses dynamic masking instead of static masking, at any training instance every sequence is masked in a different way. It removes the NSP objective from BERT and instead increases the batch size of model and reduces the number of training steps keeping the computational cost same.

DistilBERT - Distilled version of BERT [7]
Pretraining Approach: Knowledge Distillation, DM, No NSP
It introduces knowledge distillation, DistilBERT a student model is trained to mimic the behaviour of larger BERT model - teacher model. The teacher BERT model is typically a larger, well-trained model with more parameters and better performance. Further the encoder layers in the architectures are reduced from 12 to 6.

ALBERT - A Lite BERT [8]
Pretraining Approach : MLM, Parameter Reduction & NSP
ALBERT reduces the number of embedding parameters by sharing embeddings across tokens, both within a layer and across layers. In BERT, each layer has its own set of parameters, leading to a large number of parameters in deep models. ALBERT shares the parameters across layers, meaning that certain layers use the same set of parameters, reducing the model size significantly.

XLM-RoBERTa - Cross-lingual Language Model RoBERTa [9]
Pretraining Approach: MLM and Translation Language Modelling(TLM)

It is pretrained on a diverse multilingual corpus using a combination of two pretraining objectives: MLM and TLM. The MLM objective involves predicting masked tokens in a sentence, while the TLM objective encourages the model to predict translations of sentences in different languages. XLM-RoBERTa also incorporates language embeddings to distinguish between languages, and it employs parallel and adversarial training to enhance cross-lingual transfer learning.

5. Model Architecture

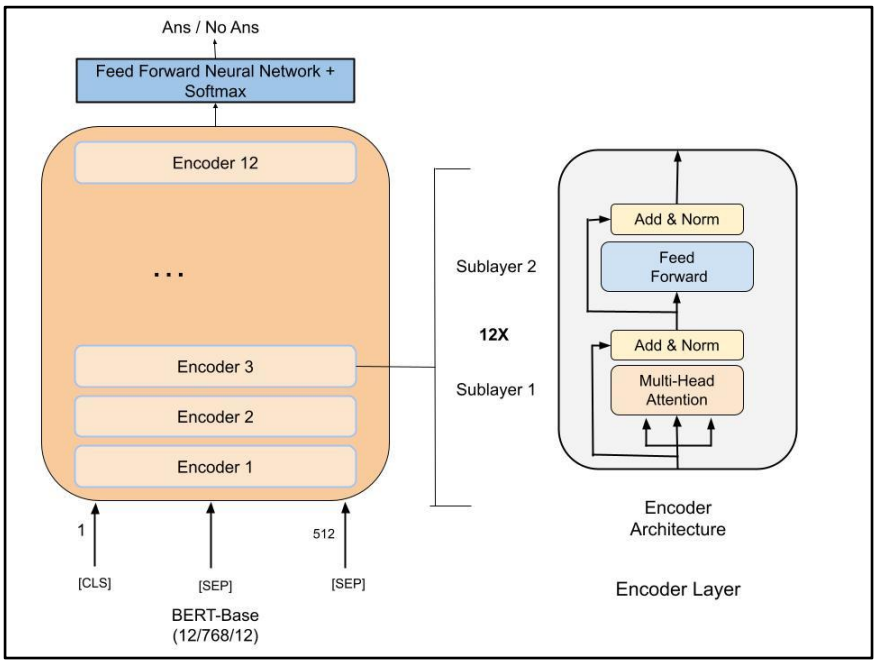


Figure 1: BERT Overview and Architectural Component

BERT is basically an Encoder stack of transformer architecture [1]. All the models mentioned in the paper follow the same architecture except for DistilBERT which have 6 encoder layers. They differ in terms of pretraining approach and every model has its unique advantages and strengths. They use the Encoder-part of the Transformer Architecture which is a deep learning architecture based on attention mechanism.

Model architecture is represented in the form of L/H/A where **L** are the number of Encoder layers, **H** is the hidden size and **A** is the number of self-attention heads.

BERT$_{\text{BASE}}$ (L=12, H=768, A=12, Total Parameters=110M)

Input/Output Representations: The input representations which include question and context are packed together in a single sequence after tokenization. The first token of every sequence is always a special classification token ([CLS]). Question is passed after the [CLS] token and it is differentiated from the context using a special separator token ([SEP]).The output representation for question answering involves predicting the start and end positions of the answer span within the passage.

As represented, BERT-base has 12 encoder layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network [3]. We employ a residual connection around each of the two sub-layers, followed by layer normalization.

Multi-head attention layer: The encoder's inputs first flow through a multi-head attention layer that consists of several self-attention layers running in parallel. The attention mechanism allows the model to focus/or pay attention on specific parts of the input data that are important to the current prediction or decision. It generates attention vectors for all words in the sequence before passing the embeddings to the model.

Feed-forward neural network: The outputs of the multi-head attention layer are fed to this layer.The exact same feed-forward network is independently applied to each position. Both the self-attention and feed-forward sub-layers are followed by layer normalization and residual connections. Layer normalization helps stabilize the learning process, and residual connections mitigate the vanishing gradient problem and enable efficient training.

6. Implementation using Fine-tuning approach

Fine-tuning is the second step in transfer learning. The main goal of transfer learning is to transfer knowledge acquired by training on one task to another task. Fine-tuning, also referred to as domain adaptation, is a process in which the pretrained model is trained with a limited labelled task-specific dataset. We fine-tune the pretrained model for QA by using the SQuAD 2.0 dataset. The pretrained model can be fine-tuned for any task like sentimental analysis, token classification and name entity recognition. For training the models we use open-source NVIDIA T4 GPU as hardware accelerator which is provided by Google Colaboratory. Training is done by default on 12 GB RAM and 78GB of disk space provided by google.

6.1 Preprocessing

1. Loading the Tokenizer: Each model has a corresponding tokenizer associated with it. For this we need to import the Autotokenizer module of the transformer library and load the tokenizer using the Autotokenizer.from_pretrained command.The tokenization used is wordpiece tokenization which has a vocabulary of 30K.

2. The Tokenizer function: Passing text to the tokenizer function encodes the text into tensors. First the text is divided into subsequent tokens, each of the tokens is allotted with an input_id and a attention_mask. Special tokens like [CLS] and [SEP] tokens are inserted in the sequence and input-ids are then converted into tensors.

3. Preprocessing hyperparameters initialized include:
    1. max_length = 384
    2. doc_stride = 128

4. Preprocessing parameters: The tokenizer function has 3 important parameters for preprocessing which include padding, truncation, return_tensors. If we want to pass input to the model the inputs should be in a uniform shape. Padding and Truncation thus helps us to create rectangular tensors from batches of varying lengths. Return tensors return the actual tensors fed to the model.

5. Map function: Preprocessing is applied on the entire dataset using the datasets map function in which by default batch size is 1000.

6. Answer Retrieval: For extracting the answers from the context we create a program which will iterate through the whole context and return the start position and end position labels of the answer. If the answer is not present in the context the model simply returns a classification token.

6.2 Training

1. Loading the Model using Autoclass: Training the model starts by importing the model using AutoModelForQuestionAnswering module and downloading the model using the .from_pretrained() command.

2. Hyperparameter Optimization: We perform hyperparameter optimization by first defining a search space, choosing the initial set of hyperparameters, and evaluating the model's performance. We then run an exhaustive search method for all sets of parameters and select the best parameter based on the results.

3. After performing hyperparameter optimization we choose the best set of parameters considering the resource limitations and evaluation results. The hyperparameter value used for training are:
    1. Batch size = 16
    2. Epochs = 3
    3. Learning Rate = 3e-5
    4. Weight Decay = 0.01

4. Passing Training Arguments:In training arguments we pass all the hyperparameter values, training settings which include evaluation and save strategy and the name of the model to be saved.

5. Data Collator to batch processed examples before passing to the Trainer():The Data Collator prepares the processed data from the map function for training. Data collator is a utility that takes care of batching, padding, and collating the input data to format it properly before passing the data from training.

6. Configuring and compiling the Trainer:Trainer function configures all the things required for training which includes model, training arguments, processed training and validation data and the tokenizer.

7. Start training and saving the model:We start training using trainer.train method and save the model at completion using trainer.save method. After starting the training we can see its progress using a progress bar which shows the number of

training steps completed, time taken and it also shows the training loss and validation loss at each step.

6.3 Evaluation

The 2 most prominent metrics used for evaluating QA systems are :
1. Exact-Match(EM)
2. F1-score

In order to evaluate the fine-tuned models we use the evaluate library and the ipython-autotime library. For the same purpose we import the evaluator module and pass the validation dataset and model checkpoint to the task_evaluator.compute function. The function takes a few minutes to iterate through the dataset. It iterates through the validation dataset and calculates the EM and F1-score of each question answer pair, the total score is then computed by averaging over the individual example scores.

7.Comparative Study

7.1 Comparative Analysis

In this section we perform comparative analysis of the performance and time taken by the models for execution. In order to evaluate the fine-tuned models we use the evaluate library and the ipython-autotime library. The Ipython-autotime library displays the time taken to complete evaluation for each fine-tuned model. Evaluation is done on the SQuAD v2 dataset and the hardware accelerator used is GPU T4 which is provided by Google Colaboratory.

Models	**BERT**	**RoBERTa**	**DistilBERT**	**ALBERT**	**XLM-RoBERTa**
Training Time	2:54:12	3:00:06	1:30:41	3:37:20	3:27:28
Evaluation Time	00:03:09	00:01:52	00:03:24	00:03:08	00:03:29

Table 2: Comparison table of time taken to complete training and evaluation of Models

The 2 most prominent metrics used for evaluating QA systems are :
1. Exact-Match(EM)
2. F1-score

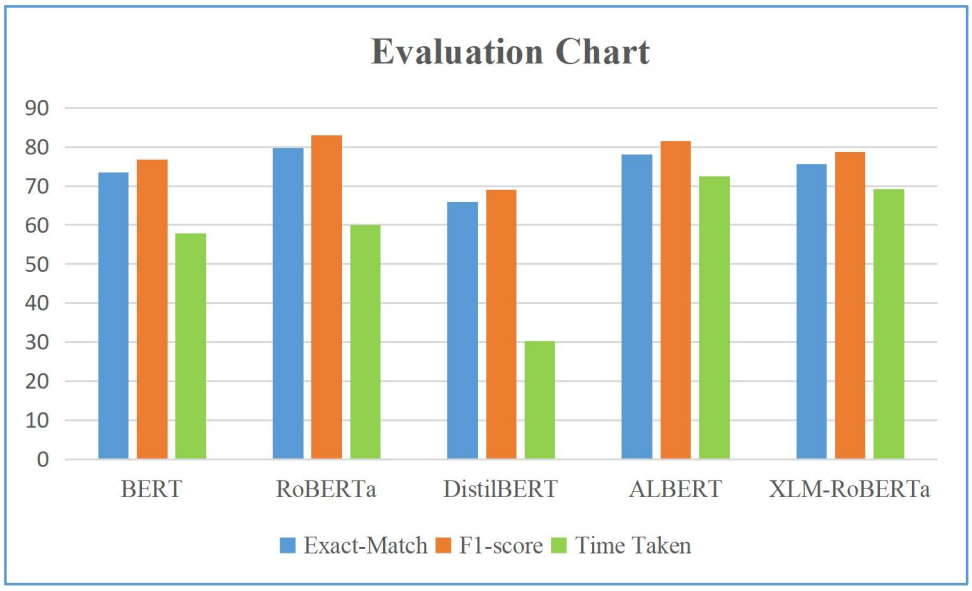
EM as the name suggests, gives a score of 1 if all the characters in the Predicted Output matches with all the characters of Actual Output, else it gives a score of 0. If the predicted output differs by a single character then, EM score = 0. F1-score is used for classification problems. It is calculated by first calculating precision and recall and then applying the formula, harmonic mean of precision and recall.

EM and F1-score are calculated individually for each question-answer pair and total score is computed for a model by averaging over the individual example scores. For performing the evaluation we use the validation dataset of SQuAD 2.

Model	BERT	RoBERTa	DistilBERT	ALBERT	XLM-RoBERTa
EM	73.50	79.72	65.88	78.13	75.52
F1	76.79	83.05	68.98	81.55	78.73

Table 3: Comparison table of performances of models on EM and F1-score

7.2 Inference



Based on the evaluation chart 1 we can conclude the following points:

1. **RoBERTa** gives the highest EM(79.72) and F1-score(83.05) hence it can be used if we want the best prediction results.

2. If we want faster finetuning we can use **DistilBERT.** It has finetuning time almost 50% less than BERT, although it reduces the performance by some percent.

3. **ALBERT** is computationally inexpensive and also requires less memory to run and it gives score(81.55) almost equal to that of RoBERTa.

4. **XLM-RoBERTa** gives F1-score of (78.73) which is almost similar to that of BERT(76.79), moreover it can be used to implement multilingual question answering giving the same performance as of BERT

5. If we do not need any performance advantages given by BERT variants, using the foundation model **BERT** can be a good choice.

8.Conclusion and Future Work

In this research paper, we conducted an in-depth analysis and review of the BERT model family, which has revolutionized the field of NLP since its introduction. We explored various variants and advancements within the BERT family, including RoBERTa, ALBERT, DistilBERT and XLM-RoBERTa. The task implemented in the paper is Answer Retrieval QA using a reading comprehension dataset. We have finedtuned BERT on question-answering and further evaluated the finetuned model using evaluation metrics. Paper further shows comparison of performances of all the mentioned models and conclusions derived on the basis of results.

Future work for BERT-based models include reducing dependency on hardware accelerators like GPUs and TPUs. This includes either working towards making hardware accelerators open-source or reducing the total parameters of the encoder model while maintaining its efficiency and performance. Research can be done on exploring new architecture for the models and further hyperparameter tuning which will result in better performance. Although using BERT reduces the carbon footprint because of transfer learning we can still improve further in terms of minimizing the carbon footprint.

References

1. Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In arXiv:1810.04805v2

2. Biswal, A. (2023). Recurrent Neural Network(RNN) tutorial: types, examples, LSTM and more. Simplilearn.com.

3. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin. (2017). Attention Is All You Need. In arXiv:1706.03762v5

4. Zhen Wang. (2022). Modern Question Answering Datasets and Benchmarks: A Survey. In arXiv:2206.15030v1

5. The Stanford question answering Dataset.
(n.d.). https://rajpurkar.github.io/SQuAD-explorer/

6. Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, Veselin Stoyanov. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. In arXiv:1907.11692v1

7. Victor Sanh, Lysandre Debut, Julien Chaumond, Thomas Wolf. (2020). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. In arXiv:1910.01108v4

8. Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, Radu Soricut. (2020). ALBERT: A Lite BERT For Self-supervised Learning Of Language Representations. In arXiv:1909.11942v6

9. Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, Veselin Stoyanov. (2020). Unsupervised Cross-lingual Representation Learning at Scale. In arXiv:1911.02116v2

10. Alammar, J. (n.d.-b). The illustrated transformer. https://jalammar.github.io/illustrated-transformer/

11. Alammar, J. (n.d.). The Illustrated BERT, ELMo, and co. (How NLP Cracked Transfer Learning). https://jalammar.github.io/illustrated-bert/